

33. INGENIERÍA DE REQUISITOS. VERIFICACIÓN. VALIDACIÓN. ESPECIFICACIÓN DE REQUISITOS. GESTIÓN DE REQUISITOS.

33- Ingeniería de requisitos. Verificación. Validación. Especificación de requisitos. Gestión de requisitos.

INDICE

- 33.1 Ingeniería de requisitos
 - 33.1.1 Tipos de requisitos
- 33.2 Identificación de los requisitos del software
 - 33.2.1 Entrevistas
 - 33.2.2 JAD (Joint Application Design)
 - 33.2.3 Prototipos
 - 33.2.4 Análisis de factores críticos de éxito
 - 33.2.5 Brainstorming
 - 33.2.6 Escenarios y casos de uso
 - 33.2.7 Etnografía
- 33.3 Verificación - Validación
- 33.4 Especificación de Requisitos
 - 33.4.1 IEEE/ANSI 830-1998
- 33.5 Gestión de requisitos
 - 33.5.1 Planificación de la gestión requisitos.
 - 33.5.2 Gestión del cambio.

35.1 Ingeniería de requisitos

En la ingeniería de software, la Ingeniería de requisitos comprende todas las tareas relacionadas con la determinación de las necesidades o de las condiciones a satisfacer para un software nuevo o modificado, tomando en cuenta los diversos requisitos de los inversores. Así, los requisitos se generan a partir de la interacción entre los usuarios y los ingenieros del

software, representando las características del sistema a construir, es decir, las necesidades de los usuarios.

Hay múltiples definiciones del término requisito, algunas de ellas son:

- Según IEEE un **requisito** es:
 - o Una condición o capacidad necesitada por un usuario para resolver un problema o alcanzar un objetivo.
 - o Una condición o capacidad que debe cumplir o poseer un sistema o un componente del mismo para satisfacer un contrato, un estándar, una especificación, u otro documento impuesto de una manera formal.
 - o Una representación documentada de una condición o capacidad tal como las expresadas en los dos puntos anteriores.
- Desde el punto de vista del usuario, los requisitos son las condiciones o capacidades necesarias para que un usuario pueda resolver un problema o alcanzar un objetivo
- Desde el punto de vista del equipo de desarrollo, los requisitos son las capacidades o condiciones que debe reunir un sistema para satisfacer un contrato, estándar o cualquier otro documento impuesto formalmente.

En resumen, los requisitos son las características que debe cumplir el sistema para que cubra las necesidades de los usuarios. Muchas veces se habla de requerimientos en vez de requisitos. Esto se debe a una mala traducción del inglés. La palabra *requirement* debe ser traducida como requisito, mientras que requerimiento se traduce al inglés como *request*.

Sommerville divide el proceso de ingeniería de requisitos en cuatro subprocesos que son la evaluación de si el sistema es útil para el negocio (**estudio de viabilidad**); el **descubrimiento de requisitos** (obtención y análisis); la transformación de estos requisitos en formularios estándar

(**especificación**), y la verificación de que los requisitos realmente definen el sistema que quiere el cliente (**validación**).

Para Thayer, *“La ingeniería de requisitos proporciona el mecanismo apropiado para entender lo que el cliente quiere, analizar las necesidades, evaluar la factibilidad, negociar una solución razonable, especificar la solución sin ambigüedades, validar la especificación, y administrar los requisitos conforme se transforman en un sistema operacional”* y establece las siguientes fases para el proceso:

- **Inicio.** Se establece una comprensión básica del problema por parte de los analistas.
- **Obtención.** Se obtienen los requisitos del software mediante la interacción entre los analistas y el cliente.
- **Elaboración.** Se refina la información obtenida en el paso anterior y se enfoca a la construcción de un modelo de análisis que represente al sistema a construir.
- **Negociación.** Hay requisitos que no son implementables o son difíciles de hacerse, por esta razón los analistas negocian estos requisitos para llegar a un entendimiento y lograr un sistema factible de desarrollarse en un plazo y coste.
- **Especificación.** Se confecciona un conjunto de documentos (descripciones en lenguaje natural, diagramas, etc.) que definan lo que el sistema debe hacer.
- **Validación.** Se examina la especificación para asegurar que todos los requisitos software se han establecido de una manera precisa, que no hay inconsistencias, omisiones ni errores, además de cumplirse los estándares de calidad establecidos para el proyecto.
- **Gestión de Requisitos.** Esta actividad permite tratar con el inevitable problema de los cambios de especificaciones, identificando, controlando y determinando el impacto del cambio de requisitos en el resto.

Otros autores modifican el número de etapas y las dividen en:

- **Educción de Requisitos.** En ella los analistas obtienen las necesidades del cliente a partir de todas las fuentes de información que tienen disponibles (documentación, entrevistas, estudio de los procesos de la organización, etc.). Términos equivalentes usados por los ingenieros de software para esta actividad son Extracción de Requisitos, Identificación de Requisitos, Determinación de Requisitos, etc. Vemos que estas se corresponderían con las dos primeras del modelo anterior.
- **Análisis de Requisitos:** se procede a trabajar sobre los requisitos educidos en el paso anterior. Se estudian los requisitos en busca de conflictos e incoherencias, implicaciones, información no obtenida y aspectos no resueltos. Después se clasifican, evalúa su viabilidad y se integran los nuevos requisitos con los ya existentes. El objetivo final es lograr una lista de requisitos que defina las necesidades del cliente. Corresponderían con las fases de Elaboración y Negociación del modelo anterior.
- **Representación de los Requisitos.** Actividad en la que se representan los requisitos de una o más formas, utilizando para ello diferentes técnicas como por ejemplo el lenguaje formal, el lenguaje natural, representaciones gráficas, etc. Para la representación existen múltiples técnicas, las más usadas son, entre otras, los Diagramas de Flujo de Datos, Modelo Entidad Relación, Casos de Uso o Diagramas de Clases. Una vez que están los requisitos representados, es necesario que se reúnan los diversos participantes en el desarrollo para revisarlos y aprobarlos. El producto final con el que culmina esta fase es la Especificación de Requisitos Software, en donde está descrito con exactitud todo lo que el sistema debe hacer. Corresponde a las fases de Especificación y Validación del modelo anterior.

- **Validación de Requisitos** Se procede a definir una serie de criterios y técnicas que permitirán, cuando el sistema esté construido, comprobar que éste cumple los requisitos.

Independientemente del modelo que usemos, lo importante es tener claro que el objetivo de la ingeniería de requisitos es determinar con claridad y precisión qué es lo que hay que hacer y para ello será necesario identificar los requisitos clave.

Para pensar en la importancia de los requisitos en la Ingeniería del Software, algunos estudios reflejan que el 45 por 100 de los errores tienen su origen en los requisitos y en el diseño preliminar, otros señalan que el 56 por 100 de los errores que tienen lugar en un proyecto software se deben a una mala especificación de requisitos, etc.

Los factores principales que conducen al fracaso en los proyectos software son: la falta de comunicación con los usuarios, los requisitos incompletos y los cambios en los requisitos; y la evidencia demuestra que los requisitos contienen demasiados errores, que muchos de estos errores no se detectan al principio, pero podrían ser detectados, y que no detectar estos errores incrementa grandemente los costes del proyecto y su duración. La consecuencia es que el sistema no satisfará a los usuarios, se producirán desacuerdos entre usuarios y desarrolladores, y se gastará tiempo y dinero en construir un sistema equivocado.

Quizá no sea posible elaborar los requisitos con absoluta perfección, pero se debe intentar minimizar el impacto de los errores en los requisitos y organizar mejor las tareas relacionadas con ellos. Para remediar esta situación es para lo que surge la **Ingeniería de Requisitos**.

33.1 Tipos de requisitos

La mayoría de los autores distinguen entre:

- **Requisitos funcionales:** Son declaraciones de los servicios que debe proporcionar el sistema, de la manera en que éste debe reaccionar a entradas particulares y de cómo se debe comportar en situaciones concretas. En algunos casos, los requisitos funcionales de los sistemas también pueden declarar explícitamente lo que el sistema no debe hacer. Los requisitos funcionales de un sistema describen lo que el sistema debe hacer.
- **Requisitos no funcionales:** Son restricciones de los servicios o funciones ofrecidos por el sistema. Incluyen restricciones de tiempo, sobre el proceso de desarrollo y estándares. Como su nombre sugiere, no se refieren directamente a las funciones específicas que proporciona el sistema, sino a las propiedades de éste como la fiabilidad, el tiempo de respuesta y la capacidad de almacenamiento. Pueden venir de las características requeridas del software (requisitos del producto), de la organización que desarrolla el software (requisitos organizacionales) o de fuentes externas.
 - o **Requisitos del producto.** Estos requisitos especifican el comportamiento del producto. Algunos ejemplos son los requisitos de rendimiento en la rapidez de ejecución del sistema y cuánta memoria se requiere; los requisitos de fiabilidad que fijan la tasa de fallos para que el sistema sea aceptable; los requisitos de portabilidad, y los requisitos de usabilidad.
 - o **Requisitos organizacionales.** Estos requisitos se derivan de políticas y procedimientos existentes en la organización del cliente y en la del desarrollador. Algunos ejemplos son los estándares en los procesos que deben utilizarse; los requisitos de implementación, como los lenguajes de programación o el método de diseño a utilizar, y los requisitos de entrega que

especifican cuándo se entregará el producto y su documentación.

- o **Requisitos externos.** Este gran apartado incluye todos los requisitos que se derivan de los factores externos al sistema y de su proceso de desarrollo. Éstos pueden incluir los requisitos de interoperabilidad que definen la manera en que el sistema interactúa con sistemas de otras organizaciones; los requisitos legislativos que deben seguirse para asegurar que el sistema funcione dentro de la ley, y los requisitos éticos. Estos últimos son puestos en un sistema para asegurar que será aceptado por sus usuarios y por el público en general.

Sommerville distingue entre **requisitos del usuario**, que son declaraciones, en lenguaje natural y en diagramas, de los servicios que se espera que el sistema proporcione y de las restricciones bajo las cuales debe funcionar, y **requisitos del sistema**, que establecen con detalle las funciones, servicios y restricciones operativas del sistema. Estos diferentes niveles de especificación serían de utilidad debido a que comunican la información del sistema a diferentes tipos de lectores.

33.2 Identificación de los requisitos del software

La educación, identificación o determinación de requisitos es el paso durante el cual los requisitos del software son obtenidos de fuentes tales como: la gente implicada (usuarios, clientes, expertos en la materia, etc.), las necesidades que ha de satisfacer el sistema, el entorno físico que rodea al sistema, el entorno organizacional, etc.

Los problemas de la obtención de requisitos se pueden agrupar en tres categorías:

- 1- Problemas de alcance, ya que los requisitos pueden implicar demasiada o muy poca información.

- 2- Problemas de comprensión, como consecuencia de una pobre comunicación entre usuario y analista. En este caso los requisitos obtenidos son ambiguos, incompletos, inconsistentes e incorrectos, porque no responden a las verdaderas necesidades de los usuarios.
- 3- Problemas de volatilidad, ya que los requisitos evolucionan con el tiempo. En efecto, a medida que avanza el desarrollo del sistema, las necesidades del usuario pueden madurar a causa del conocimiento adicional fruto del desarrollo, o de necesidades del entorno o de la organización no previstas.

La solución al primer problema pasa por determinar claramente el contexto del sistema, es decir, los límites y objetivos del mismo. Si no se contempla el contexto donde va a funcionar el sistema, se puede llegar a requisitos incompletos, no verificables, innecesarios y no utilizables. La solución del segundo es que exista una buena comunicación entre usuarios, desarrolladores y clientes, a fin de que los requisitos se puedan escribir de manera que permitan, tanto que el desarrollador pueda distinguir si dichos requisitos se pueden implementar, como que el personal de control pueda comprobar si la implementación cumple con los requisitos. Por último la solución al tercer problema es incorporar estos cambios a los requisitos originales, pues si no, éstos serán incompletos e inconsistentes con la nueva situación.

Son numerosas las estrategias y técnicas que se han desarrollado para la obtención de los requisitos. Las más importantes las veremos a continuación.

33.2.1 Entrevista

Se entiende por entrevista el encuentro que se realiza “cara a cara” entre un usuario y la persona responsable de obtener la información. Para realizar la entrevista solo es necesario designar a las personas que deben

participar en ella y determinar el lugar en el que poder llevarla a cabo. Es importante identificar a qué tipo de perfil va dirigida la entrevista, a quiénes se va a entrevistar y cuál es el momento más oportuno, con el fin de evitar situaciones embarazosas y conseguir que la entrevista sea eficaz y productiva.

Como paso previo a la realización de la entrevista se deben tener en cuenta una serie de reglas generales o directrices básicas:

- Desarrollar un plan global de la entrevista.
- Asegurarse de que se cuenta con la aprobación para hablar con los usuarios.
- Preparar la entrevista previamente.
- Realizar la entrevista.
- Consolidar el resultado de la entrevista.

Además, es conveniente planificar las entrevistas estudiando la secuencia en que se van a llevar a cabo, en función de los distintos perfiles implicados y las relaciones existentes entre los entrevistados. Según la información a obtener y dependiendo de las distintas fuentes que pueden proporcionarla, puede ser necesario realizar una entrevista conjunta con varias personas. Durante la preparación de la entrevista es imprescindible remitir al usuario un guión previo sobre los puntos a tratar, para que pueda estudiarlo con tiempo y solicitar la información que estime conveniente para la entrevista. Se debe pensar bien el tipo de guión, según el perfil y las responsabilidades del entrevistado y su extensión, de forma que se pueda conseguir la suficiente información, sin provocar rechazo en el entrevistado. Si se considera apropiado se pueden utilizar herramientas automatizadas.

Una vez que se dispone de la aprobación para hablar con los usuarios, se hace la convocatoria de la entrevista enviando la información oportuna y fijando los objetivos, el método de trabajo que se va a seguir y el tiempo del que se dispone.

Para realizar la entrevista, es importante hacer un resumen general de los temas a tratar, utilizar un estilo apropiado y crear desde su inicio un clima de confianza entre los asistentes. Es posible que el entrevistado se resista a aportar información, siendo útil en estos casos utilizar técnicas específicas de comunicación.

Antes de finalizar la entrevista es importante que el entrevistador sintetice las conclusiones y compruebe que todos los asistentes están de acuerdo, dejando siempre abierta la posibilidad de volver a contactar para aclarar temas que surjan al estudiar la información recopilada.

Finalmente, el responsable depura y consolida el resultado de las entrevistas, elaborando un informe de conclusiones. En algunos casos puede ser conveniente elaborar un acta que refleje estas conclusiones y remitirla a los entrevistados con el objetivo de asegurar que se han comprendido bien las especificaciones dadas.

33.2.2 JAD (Joint Application Design)

Las características de una sesión de trabajo tipo JAD se pueden resumir en los siguientes puntos:

- Se establece un equipo de trabajo cuyos componentes y responsabilidades están perfectamente identificados y su fin es conseguir el consenso entre las necesidades de los usuarios y los servicios del sistema en producción.
- Se llevan a cabo pocas reuniones, de larga duración y muy bien preparadas.
- Durante la propia sesión se elaboran los modelos empleando diagramas fáciles de entender y mantener, directamente sobre herramientas CASE.

- Al finalizar la sesión se obtienen un conjunto de modelos que deberán ser aprobados por los participantes.

Es importante definir claramente el perfil y las responsabilidades de los participantes de una sesión JAD. Se pueden distinguir los siguientes perfiles:

- Moderador (líder) con amplios conocimientos de la metodología de trabajo, dinámica de grupos, psicología del comportamiento, así como de los procesos de la organización objeto del estudio.
- Promotor, persona que ha impulsado el desarrollo.
- Jefe de proyecto, responsable de la implantación del proyecto.
- Especialista en modelización, responsable de la elaboración de los modelos en el transcurso de la sesión.
- Desarrolladores, aseguran que los modelos son correctos y responden a los requisitos especificados.
- Usuarios, responsables de definir los requisitos del sistema y validarlos.

Para llevar a cabo una sesión JAD, es necesario realizar una serie de actividades antes de su inicio, durante el desarrollo y después de su finalización. Estas actividades se detallan a continuación:

- **Inicio:** se define el ámbito y la estructura del proyecto, los productos a obtener, se prepara el material necesario para la sesión, se determina el lugar donde se van a llevar a cabo, se seleccionan los participantes y se sugiere una agenda de trabajo.
- **Desarrollo:** se identifican las salidas del proyecto y se debe conseguir el consenso entre los participantes de modo que se materialice en los modelos.
- **Finalización:** se valida la información de la sesión y se generan los productos de la metodología de trabajo propuesta. Si fuera necesario se integran los productos de salida.

33.2.3 Prototipado

Los prototipos son sistemas software que sólo implementan una parte del sistema. Normalmente, los prototipos se emplean para obtener requisitos del usuario cuando éstos no están completamente claros. Es mucho más fácil que el usuario entienda y apruebe un sistema, que aunque es reducido pero también es operativo y puede interaccionar con él, a revisar una larga lista de texto con los requisitos que describen dicho sistema. Al fin y al cabo, como se suele decir, el prototipo permite salvar la situación siguiente: “No se lo que quiero, pero lo sabré cuando lo vea”. Y en efecto, será mucho más fácil para un usuario saber si lo que quiere es lo que tiene delante y que puede interaccionar con él, que si es un montón de hojas con todos los requisitos escritos uno a uno.

La aplicación de la técnica del prototipado consta de los siguientes pasos:

- 1- Estudio preliminar de los requisitos del usuario.
- 2- Proceso iterativo consistente en:
 - a. Construir un prototipo.
 - b. Evaluarlo con los usuarios.
 - c. Formular nuevos requisitos.
 - d. Desechar el prototipo.

Los aspectos clave en el diseño de prototipos son: la identificación de los usuarios a los que va dirigido, teniendo en cuenta que debe responder a diferentes individualidades, con distintos conocimientos y habilidades, qué funciones tienen asignadas, y qué tipo de información requerirán para llevar a cabo dichas funciones.

33.2.4 Análisis de factores críticos de éxito.

Esta técnica consistente en identificar y concentrarse en un pequeño conjunto de factores críticos de los que depende el éxito y efectividad del sistema. Aunque la identificación de los factores críticos correctos es, a veces, una labor compleja, esta técnica es bastante útil cuando el sistema es técnicamente complejo.

33.2.5 Brainstorming

Esta técnica se usa principalmente para la generación de ideas en los casos en los que la generación de éstas no es obvia. Se trata de una técnica sencilla en la que se reúnen en grupo de 4 a 10 personas para generar ideas, sin restricciones, en un ambiente libre de críticas. Uno de los principales puntos fuertes de la técnica es que ideas que en un principio pueden ser descabelladas tienen cabida. Primero se proporcionan las ideas y en una segunda vuelta se refinan. El Brainstorming permite aportar a la obtención de requisitos los siguientes aspectos:

- **Genera múltiples puntos de vista de un problema.** Cada uno da su visión del problema, que no tiene porqué ser la misma. Una vez vistos todos los puntos de vista se puede atacar el problema de una forma más efectiva.
- **Formular un problema de distintas formas.** Cada participante puede ver el problema desde una óptica distinta, por lo que a la hora de formularlo lo hará de una forma distinta. Estas diferencias pueden aportar gran valor a la hora del estudio del problema, identificando los puntos clave del problema de cada participante.

La técnica de Brainstorming, para que sea efectiva, debe ser previamente preparada. Las fases por las que pasa la ejecución de la técnica son:

- 1- **Fase de Preparación:** Se identifican los participantes de la sesión (clientes, usuarios, analistas, etc.), se designa un líder que lleva la sesión, se planifica la sesión y se busca una sala adecuada.

- 2- **Fase de Generación:** Se comienza exponiendo las ideas de forma libre, por turnos o espontáneamente. Las ideas se van apuntando en una pizarra que todos puedan ver.
- 3- **Fase de Consolidación:** Se revisan las ideas obtenidas en el paso anterior, se clarifican si no están claras, se rescriben si es necesario, se descartan las que no son utilizables, se discuten las restantes y se priorizan.

33.2.6 Escenarios y Casos de uso

Normalmente, las personas encuentran más fácil dar ejemplos de la vida real que descripciones abstractas. Los **escenarios** pueden ser especialmente útiles para agregar detalle a un esbozo de la descripción de requisitos. Son descripciones de ejemplos de las sesiones de interacción. Cada escenario abarca una o más posibles interacciones. El escenario comienza con un esbozo de la interacción y, durante la obtención se agregan detalles para crear una descripción completa de esta interacción. De forma general, un escenario puede incluir:

- 1- Una descripción de lo que esperan el sistema y los usuarios cuando el escenario comienza.
- 2- Una descripción del flujo normal de eventos en el escenario.
- 3- Una descripción de lo que puede ir mal y cómo manejarlo.
- 4- Información de otras actividades que se podrían llevar a cabo al mismo tiempo.
- 5- Una descripción del estado del sistema cuando el escenario termina.

Los escenarios se pueden redactar como texto, complementados por diagramas, fotografías de las pantallas, etcétera. De forma alternativa, se puede adoptar un enfoque más estructurado, como los escenarios de evento o los casos de uso.

Los casos de uso son una técnica que se basa en escenarios para la obtención de requisitos. En su forma más simple, un caso de uso identifica el tipo de interacción y los actores involucrados. Los actores en el proceso se representan como figuras delineadas, y cada clase de interacción se representa como una elipse con su nombre. El conjunto de casos de uso representa todas las posibles interacciones a representar en los requisitos del sistema. Actualmente se han convertido en una característica fundamental de la notación de UML, que se utiliza para describir modelos de sistemas orientados a objetos.

Los escenarios y los casos de uso son técnicas eficaces para obtener requisitos para los puntos de vista de los interactuadores, donde cada tipo de interacción se puede representar como un caso de uso. Sin embargo, debido a que se centran en las interacciones, no son tan eficaces para obtener restricciones y requisitos de negocio y no funcionales de alto nivel de puntos de vista indirectos o para descubrir requisitos del dominio.

33.2.7 Observación directa e investigación contextual (Etnografía)

La **etnografía** es una técnica de observación que se puede utilizar para entender los requisitos sociales y organizacionales. Un analista se sumerge por sí solo en el entorno laboral donde se utilizará el sistema. Observa el trabajo diario y anota las tareas reales en las que los participantes están involucrados. El valor de la etnografía es que ayuda a los analistas a descubrir los requisitos implícitos que reflejan los procesos reales más que los formales en los que la gente está involucrada.

La etnografía es especialmente efectiva para descubrir dos tipos de requisitos:

- Los requisitos que se derivan de la forma en la que la gente trabaja realmente más que de la forma en la que las definiciones de los procesos establecen que debería trabajar.

- Los requisitos que se derivan de la cooperación y conocimiento de las actividades de los demás.

Los estudios etnográficos pueden revelar los detalles de los procesos críticos que otras técnicas de obtención de requisitos a menudo olvidan. Sin embargo, puesto que se centran en el usuario final, este enfoque no es apropiado para descubrir los requisitos organizacionales o del dominio. Los estudios etnográficos no siempre pueden identificar nuevas propiedades que se deban agregar al sistema. Por lo tanto, la etnografía no es un enfoque completo para la obtención de requisitos por sí mismo, y debe utilizarse para complementar otros enfoques, como el análisis de casos de uso.

33.3 Verificación - Validación

La **validación** de requisitos trata de mostrar que éstos realmente definen el sistema que el cliente desea. La validación de requisitos es importante debido a que los errores en los requisitos pueden conducir a importantes costes al repetir el trabajo cuando son descubiertos durante el desarrollo o después de que el sistema esté en uso. El coste de arreglar un problema en los requisitos haciendo un cambio en el sistema es mucho mayor que reparar los errores de diseño o los de codificación.

La razón de esto es que un cambio en los requisitos normalmente significa que el diseño y la implementación del sistema también deben cambiar y que éste debe probarse nuevamente.

Durante el proceso de validación de requisitos, se deben llevar a cabo las siguientes **verificaciones**:

- 1- **Verificaciones de validez.** Un usuario puede pensar que se necesita un sistema para llevar a cabo ciertas funciones. Sin embargo, el razonamiento y el análisis pueden identificar que se

requieren funciones adicionales o diferentes. También se deben tener en cuenta que en un mismo sistema suele haber diferentes usuarios, con diferentes puntos de vista, algunas veces contrapuestos, y que éstos deben llegar a un compromiso a la hora de definir los requisitos del sistema.

- 2- **Verificaciones de consistencia.** Los requisitos no deben contradecirse. Esto es, no debe haber restricciones o descripciones contradictorias de la misma función del sistema.
- 3- **Verificaciones de completitud.** Los requisitos deben definir todas las funciones y restricciones propuestas por el usuario del sistema.
- 4- **Verificaciones de realismo.** Utilizando el conocimiento de la tecnología existente, los requisitos deben verificarse para asegurar que se pueden implementar. Estas verificaciones también deben tener en cuenta el presupuesto y la confección de agendas para el desarrollo del sistema.
- 5- **Verificabilidad.** Para reducir la posibilidad de discusiones entre el cliente y el desarrollador, los requisitos del sistema siempre deben redactarse de tal forma que sean verificables. Esto significa que debe poder escribir un conjunto de pruebas que demuestren que el sistema a entregar cumple cada uno de los requisitos especificados.

Pueden utilizarse, en conjunto o de forma individual, varias técnicas de validación de requisitos:

- 1- **Revisiones de requisitos.** Los requisitos son analizados manual y sistemáticamente por un equipo de revisores formado por personas tanto de la organización del cliente como de la desarrolladora. Se verifican los requisitos en cuanto a anomalías y omisiones. Pueden ser informales o formales. Las informales sencillamente implican que los desarrolladores deben tratar los requisitos con tantos usuarios del sistema como sea posible. En la revisión formal de requisitos, el equipo de desarrollo debe «conducir» al cliente a través de los

requisitos del sistema, explicándole las implicaciones de cada requisito. Los revisores deben comprobar la:

- a. **Verificabilidad.** ¿Puede probarse el requisito de modo realista?
- b. **Comprensibilidad.** ¿Las personas que adquieren el sistema o los usuarios finales comprenden correctamente el requisito?
- c. **Rastreabilidad.** ¿Está claramente establecido el origen del requisito? Puede tener que volver a la fuente del requisito para evaluar el impacto del cambio. La rastreabilidad es importante ya que permite evaluar el impacto del cambio en el resto del sistema.
- d. **Adaptabilidad.** ¿Es adaptable el requisito? Es decir, ¿puede cambiarse el requisito sin causar efectos de gran escala en los otros requisitos del sistema?

Los conflictos, contradicciones, errores y omisiones en los requisitos deben ser señalados por los revisores y registrarse formalmente en el informe de revisión. Queda en los usuarios, la persona que adquiere el sistema y el desarrollador de éste negociar una solución para estos problemas identificados.

- 2- **Construcción de prototipos** En este enfoque de validación, se muestra un modelo ejecutable del sistema a los usuarios finales y a los clientes. Éstos pueden experimentar con este modelo para ver si cumple sus necesidades reales.
- 3- **Generación de casos de prueba.** Los requisitos deben poder probarse. Si las pruebas para éstos se conciben como parte del proceso de validación, a menudo revela los problemas en los requisitos. Si una prueba es difícil o imposible de diseñar, normalmente significa que los requisitos serán difíciles de implementar y deberían ser considerados nuevamente.

Es difícil demostrar que un conjunto de requisitos cumple las necesidades del usuario. Como consecuencia, rara vez se encuentran todos los problemas en los requisitos durante el proceso de validación de éstos. Es inevitable que haya cambios adicionales de requisitos para corregir las omisiones y las malas interpretaciones después de que el documento de requisitos haya sido aprobado.

33.4 Especificación de requisitos

El **documento de requisitos del software** (algunas veces denominado *especificación de requisitos del software* o **ERS**) es la declaración oficial de qué deben implementar los desarrolladores del sistema. Tiene un conjunto diverso de usuarios que va desde los altos cargos de la organización que pagan por el sistema, hasta los ingenieros responsables de desarrollar el software. Los objetivos que pretende el ERS son los siguientes:

- Proporcionar los medios de comunicación entre todas las partes implicadas en el sistema: clientes, usuarios, analistas y diseñadores.
- Servir como base para las actividades de prueba y verificación.
- Ayudar al control de la evolución del sistema software.

El documento ERS debe incluir una descripción completa y concisa de toda la interfaz externa del sistema con su entorno, incluido el resto del software, puertos de comunicación, hardware y usuarios. Es decir, debe incluir tanto los requisitos de comportamiento del sistema (funcionales), que son aquellos que definen lo que hace éste y la información que maneja, como los requisitos que no son de comportamiento, esto es, aquellos que definen los atributos del sistema según realiza su trabajo (eficiencia, fiabilidad, seguridad, etc.).

Por contra, un documento ERS no debe incluir los elementos de gestión del proyecto (planificaciones, hitos, etc.), ni el diseño, ni los planes de control

del producto (gestión de configuración, garantía de calidad, etc.). Un documento ERS debe reunir las siguientes características:

- **Correcto.** Cada requisito establecido debe representar algo requerido para el sistema.
- **No Ambiguo.** Cada requisito establecido tiene una sola interpretación.
- **Completo.** Debe incluir todo lo que el software tiene que hacer.
- **Verificable.** Se ha de poder comprobar, mediante un proceso efectivo y de coste limitado, que el producto reúne cada requisito establecido.
- **Consistente.** Cada requisito no puede estar en conflicto con otros requisitos.
- **Modificable.** La estructura y estilo del documento debe hacer fáciles los cambios.
- **Conciso,** comprensible por el usuario y organizado.
- **Referenciado.** Cada requisito debe estar calificado y debidamente referenciado.

33.4.1 IEEE/ANSI 830-1998

Varias organizaciones grandes, como el Departamento de Defensa de los Estados Unidos y el IEEE, han definido estándares para los documentos de requisitos. El estándar más ampliamente conocido es el IEEE/ANSI 830-1998 (IEEE, 1998). Este estándar IEEE sugiere la siguiente estructura para los documentos de requisitos:

1. Introducción

1.1 Propósito del documento de requisitos: Propósito y a quién va dirigido.

1.2 Alcance del producto

1.3 Definiciones, acrónimos y abreviaturas

1.4 Referencias

1.5 Descripción del resto del documento

2. Descripción general

2.1 Perspectiva del producto

2.2 Funciones del producto

2.3 Características del usuario

2.4 Restricciones generales

2.5 Suposiciones y dependencias

3. **Requisitos específicos.** Este será el grueso del documento.

3.1. Interfaces Externas: Se describirán los requisitos que afecten a la interfaz de usuario, interfaz con otros sistemas (hardware y software) e interfaces de comunicaciones.

3.2. Funciones: Esta subsección (quizá la más larga del documento) deberá especificar todas aquellas acciones (funciones) que deberá llevar a cabo el software. Si se considera necesario, podrán utilizarse notaciones gráficas y tablas, pero siempre supeditadas al lenguaje natural, y no al revés. El estándar permite organizar esta subsección de múltiples formas, y sugiere, entre otras, las siguientes:

- **Por tipos de usuario:** Distintos usuarios poseen distintos requisitos. Para cada clase de usuario que exista en la organización, se especificarán los requisitos funcionales que le afecten o tengan mayor relación con sus tareas.
- **Por objetos:** Los objetos son entidades del mundo real que serán reflejadas en el sistema. Para cada objeto, se detallarán sus atributos y sus funciones. Los objetos pueden agruparse en clases. Esta organización de la ERS no quiere decir que el diseño del sistema siga el paradigma de Orientación a Objetos.
- **Por objetivos:** Un objetivo es un servicio que se desea que ofrezca el sistema y que requiere una determinada entrada para obtener su resultado. Para cada objetivo o subobjetivo

que se persiga con el sistema, se detallarán las funciones que permitan llevarlo a cabo.

- **Por estímulos:** Se especificaran los posibles estímulos que recibe el sistema y las funciones relacionadas con dicho estímulo.
- **Por jerarquía funcional:** Si ninguna de las anteriores alternativas resulta de ayuda, la funcionalidad del sistema se especificara como una jerarquía de funciones que comparten entradas, salidas o datos internos. Se detallaran las funciones (entrada, proceso, salida) y las subfunciones del sistema. Esto no implica que el diseño del sistema deba realizarse según el paradigma de Diseño Estructurado.

3.3. Requisitos de Rendimiento: Se detallarían los requisitos relacionados con la carga que se espera tenga que soportar el sistema. Por ejemplo, el número de terminales, el número esperado de usuarios simultáneamente conectados, número de transacciones por segundo que deberá soportar el sistema, etc. También, si es necesario, se especificarán los requisitos de datos, es decir, aquellos requisitos que afecten a la información que se guardará en la base de datos. Por ejemplo, la frecuencia de uso, las capacidades de acceso y la cantidad de registros que se espera almacenar (decenas, cientos, miles o millones).

3.4. Restricciones de Diseño: Todo aquello que restrinja las decisiones relativas al diseño de la aplicación: Restricciones de otros estándares, limitaciones del hardware, etc.

3.5. Atributos del Sistema: Se detallaran los atributos de calidad del sistema: Fiabilidad, mantenibilidad, portabilidad, y, muy importante, la seguridad. Debería especificarse que tipos de usuario están autorizados, o no, a realizar ciertas tareas, y como se implementarían los mecanismos de seguridad

3.6. Otros Requisitos

4. Apéndices

5. Índice

El estándar IEEE es un marco general que se puede transformar y adaptar para definir un estándar ajustado a las necesidades de una organización en particular. La información que se incluya en un documento de requisitos debe depender del tipo de software a desarrollar y del enfoque de desarrollo que se utilice.

El documento de requisitos es fundamental cuando un desarrollador exterior está construyendo el sistema software. Sin embargo, los métodos de desarrollo ágiles sostienen que los requisitos cambian tan rápidamente que un documento de requisitos se queda desfasado en cuanto se redacta, por lo que el esfuerzo en gran parte se malgasta.

33. 5 Gestión de requisitos

Los requisitos para sistemas software grandes son siempre cambiantes. Debido a que el problema no puede definirse completamente, es muy probable que los requisitos del software sean incompletos. Durante el proceso del software, la comprensión del problema por parte de los clientes cambia, y los requisitos deben entonces evolucionar para reflejar esto. Además, una vez que un sistema se ha instalado, inevitablemente surgen nuevos requisitos. Cuando los usuarios finales trabajan con un sistema, descubren nuevas necesidades y prioridades debidas a:

- Normalmente, los sistemas grandes tienen una comunidad de usuarios diversa donde los usuarios tienen diferentes requisitos y prioridades. Los que se usaron para definir el sistema pudieron no ser los mejores.
- Las personas que pagan por el sistema y los usuarios de éste raramente son la misma persona. Los clientes del sistema imponen requisitos debido a las restricciones organizacionales y de presupuesto. Éstos pueden estar en conflicto con los requisitos de los

usuarios finales y, después de la entrega, pueden tener que añadirse nuevas características de apoyo al usuario si el sistema tiene que cumplir sus objetivos.

- El entorno de negocios y técnico del sistema cambia después de la instalación, y estos cambios se deben reflejar en el sistema. Se puede introducir nuevo hardware, puede ser necesario que el sistema interactúe con otros sistemas, las prioridades de negocio pueden cambiar con modificaciones consecuentes en la ayuda al sistema, y puede haber una nueva legislación y regulaciones que deben ser implementadas por el sistema.

La **gestión de requisitos** es el proceso de comprender y controlar los cambios en los requisitos del sistema. Es necesario poder evaluar el impacto de los cambios en los requisitos. Hay que establecer un proceso formal para implementar las propuestas de cambios. El proceso de gestión de requisitos debería empezar en cuanto esté disponible una versión preliminar del documento de requisitos, pero se debería empezar a planificar cómo gestionar los requisitos que cambian durante el proceso de obtención de requisitos.

Existen **requisitos duraderos**, que son requisitos relativamente estables que se derivan de la actividad principal de la organización y que están relacionados directamente con el dominio del sistema. Estos requisitos se pueden derivar de los modelos del dominio que muestran las entidades y relaciones que caracterizan un dominio de aplicación. Por otro lado, existen **requisitos volátiles**, que son requisitos que probablemente cambian durante el proceso de desarrollo del sistema o después de que éste se haya puesto en funcionamiento, debido a cambios del entorno, legislación,...

33.5.1 Planificación de la gestión requisitos.

Para cada proyecto, la etapa de planificación establece el nivel de detalle necesario en la gestión de requisitos. Habrá que decidir sobre:

1. **La identificación de requisitos.** Cada requisito se debe identificar de forma única de tal forma que puedan ser remitidos por otros requisitos de manera que pueda utilizarse en las evaluaciones de rastreo.
2. **Un proceso de gestión del cambio.** Éste es el conjunto de actividades que evalúan el impacto y coste de los cambios. Lo veremos en un apartado posterior.
3. **Políticas de rastreo o trazabilidad.** Estas políticas definen las relaciones entre los requisitos, y entre éstos y el diseño del sistema que se debe registrar y la manera en que estos registros se deben mantener.
4. **Selección de herramientas CASE.** La gestión de requisitos comprende el procesamiento de grandes cantidades de información sobre los requisitos. Las herramientas que se pueden utilizar van desde sistemas de gestión de requisitos especializados hasta hojas de cálculo y sistemas sencillos de bases de datos.

El concepto de trazabilidad hace referencia a la posibilidad de determinar cómo se ha llegado a un cierto elemento del software a partir de otros. Para poder llevar a cabo esta trazabilidad es especialmente importante el poder relacionar unos requisitos con otros y también relacionar requisitos con elementos del sistema a los que da lugar. Se habla de trazabilidad *hacia delante* cuando partiendo de un requisito se llega a todos los elementos que materializan dicho requisito, o *hacia atrás*, cuando partiendo de un elemento del sistema se llega al requisito que lo generó.

Un requisito se debe poder relacionar con otros requisitos similares para así evitar repeticiones en los mismos. Además, cuando se hace un cambio en uno, es más fácil localizar aquellos requisitos en los que hay relación, para ver el impacto del cambio. Por otro lado, es útil poder relacionar un

requisito con los elementos del sistema a los que éste da lugar, por si hay que cambiar un elemento del sistema, o simplemente, para poder saber qué requisitos dieron lugar a un determinado componente del sistema. Esta relación no sólo debe ser entre requisitos, sino también entre cualquier elemento que se haya derivado posteriormente, ya sea del análisis, del diseño, de la construcción, de pruebas, etc.

La gestión de requisitos necesita ayuda automatizada. Las herramientas CASE para esto deben elegirse durante la fase de planificación. Se precisan herramientas de ayuda para:

1. **Almacenar requisitos.** Los requisitos deben mantenerse en un almacén de datos seguro y administrado que sea accesible a todos los que estén implicados en el proceso de ingeniería de requisitos.
2. **Gestionar el cambio.**
3. **Gestionar el rastreo.** Las herramientas de ayuda para el rastreo permiten que se descubran requisitos relacionados. Algunas herramientas utilizan técnicas de procesamiento del lenguaje natural para ayudarle a descubrir posibles relaciones entre los requisitos.

Para sistemas pequeños, es posible que no sea necesario utilizar herramientas de gestión de requisitos especializadas, pero si es muy conveniente para sistemas grandes.

33.5.2 Gestión del cambio

La ventaja de utilizar un proceso formal para gestionar el cambio es que todos los cambios propuestos son tratados de forma consistente y que los cambios en los requisitos se hacen de forma controlada. Existen varias etapas principales en un proceso de gestión de cambio:

- **Propuesta de Cambios.** El afectado por un requisito que no le satisface debe rellenar un formulario de propuesta de cambio

indicando cual es el cambio que hay que realizar. Este cambio se remitirá al equipo de gestión de requisitos para que lo estudie.

- **Análisis de Impactos.** La propuesta de cambio se evalúa para determinar el impacto del cambio en el resto de los requisitos. Hay propuestas cuyo impacto es mínimo, y otras cuyo impacto hace que se deba modificar una parte sustancial del sistema. Además del impacto se deberá estudiar la oportunidad o necesidad del cambio. Ocurre muchas veces que a un usuario no le parece bien un determinado aspecto del sistema pero a otros si les satisface. En estos casos se debe llegar a un consenso entre todas las partes. Otras veces el cambio es interesante pero se considera que no es oportuno hacerlo en dicho momento, quizás porque se quiera lanzar el sistema cuanto antes o porque el cambio es considerable y sería mejor atacarlo en una fase posterior de desarrollo.
- **Toma de Decisiones.** Dependiendo del impacto, de la necesidad u oportunidad del cambio y de otras cuestiones específicas que puedan surgir alrededor del cambio, se debe determinar si el cambio debe ser realizado o no. Si se decide realizarlo se deben hacer las modificaciones pertinentes en el sistema para integrar el cambio con el resto, empezando por los requisitos y finalizando por las partes más avanzadas del sistema. Si por el contrario se decide no hacerlo se puede optar por dos soluciones:
 - o Rechazar el cambio, porque se considera que no tiene sentido.
 - o Posponer su realización para un futuro, se considera el cambio necesario pero el momento no es oportuno. Se pospone para cuando se pueda acometer.
- **Comunicación.** Tanto si se acepta como si no, se debe notificar los efectos de la propuesta de cambio a todos los afectados.
- **Incorporación.** Se hacen las modificaciones pertinentes que se identificaron en el análisis de impacto en los diferentes elementos afectados por el cambio.

- **Medición de la Estabilidad de los Requisitos.** Se evalúan los parámetros que definen la estabilidad de los requisitos tras las modificaciones que se hayan realizado.

Bibliografía:

Ingeniería de Software 7 Ed - Ian Sommerville

El Proceso Unificado de Desarrollo de Software - Jacobson - Booch - Rumbaugh

Métrica 3 – Técnicas y Prácticas. Ministerio de Administraciones Públicas

Especificación de Requisitos según el estándar de IEEE 830

<http://www.fdi.ucm.es/profesor/gmendez/docs/is0809/ieee830.pdf>

Autor: Hernán Vila Pérez

Jefe del Servicio de Informática. Instituto Galego de Vivenda e Solo

Vicepresidente del CPEIG